CPU Scheduling Algorithms Simulation Project

Youness Anouar Abdeljalil Otman

May 2, 2025

Abstract

This report presents the design and implementation of a comprehensive CPU scheduling simulation project. The project includes implementations of five classic scheduling algorithms: First-Come First-Served (FCFS), Shortest Job First (SJF), Priority Scheduling, Round Robin (RR), and Priority Round Robin (PRR). The report details the design choices, implementation decisions, usage scenarios, testing methodology, and performance comparisons between the algorithms. The simulation provides an interactive visual interface for educational purposes, allowing users to understand the behavior and efficiency of different scheduling approaches.

Contents

Chapter 1

Introduction

1.1 Project Overview

This project implements a CPU scheduling simulation system that demonstrates the behavior and performance of various scheduling algorithms commonly studied in operating systems courses. The simulation provides both a backend processing layer written in Python and a frontend visualization layer built with Vue.js.

1.2 Objectives

The primary objectives of this simulation project are:

- To implement multiple CPU scheduling algorithms following their theoretical specifications
- To provide an interactive visual representation of algorithm execution
- To enable performance comparison between different scheduling approaches
- To serve as an educational tool for understanding CPU scheduling concepts
- To demonstrate the practical implications of theoretical scheduling principles

1.3 Scope

The simulation covers the following aspects:

- Process creation with configurable attributes (arrival time, burst time, priority)
- Implementation of five scheduling algorithms (FCFS, SJF, Priority, Round Robin, Priority Round Robin)
- Visualization of process execution using Gantt charts
- Calculation and display of performance metrics
- Interactive step-by-step execution for detailed understanding
- Comparison tools for evaluating algorithm efficiency

Chapter 2

System Architecture

2.1 Overall Design

The project follows a client-server architecture with a clear separation between the computational backend and presentation frontend:

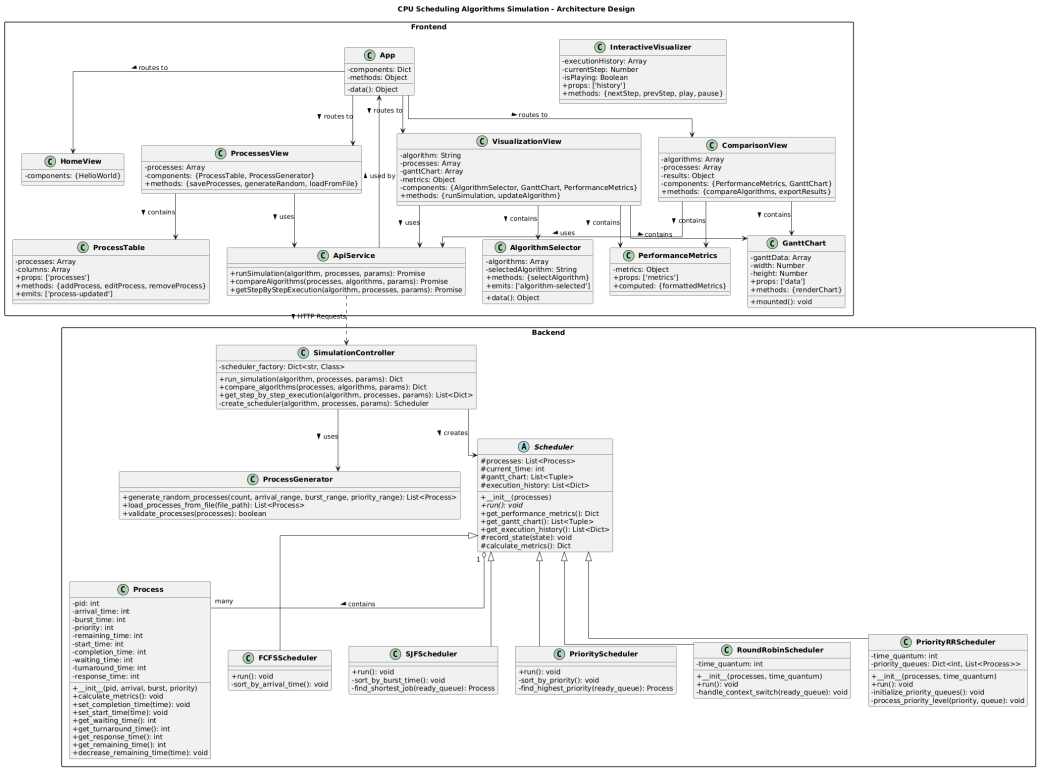

Figure 2.1: System Architecture Class Diagram

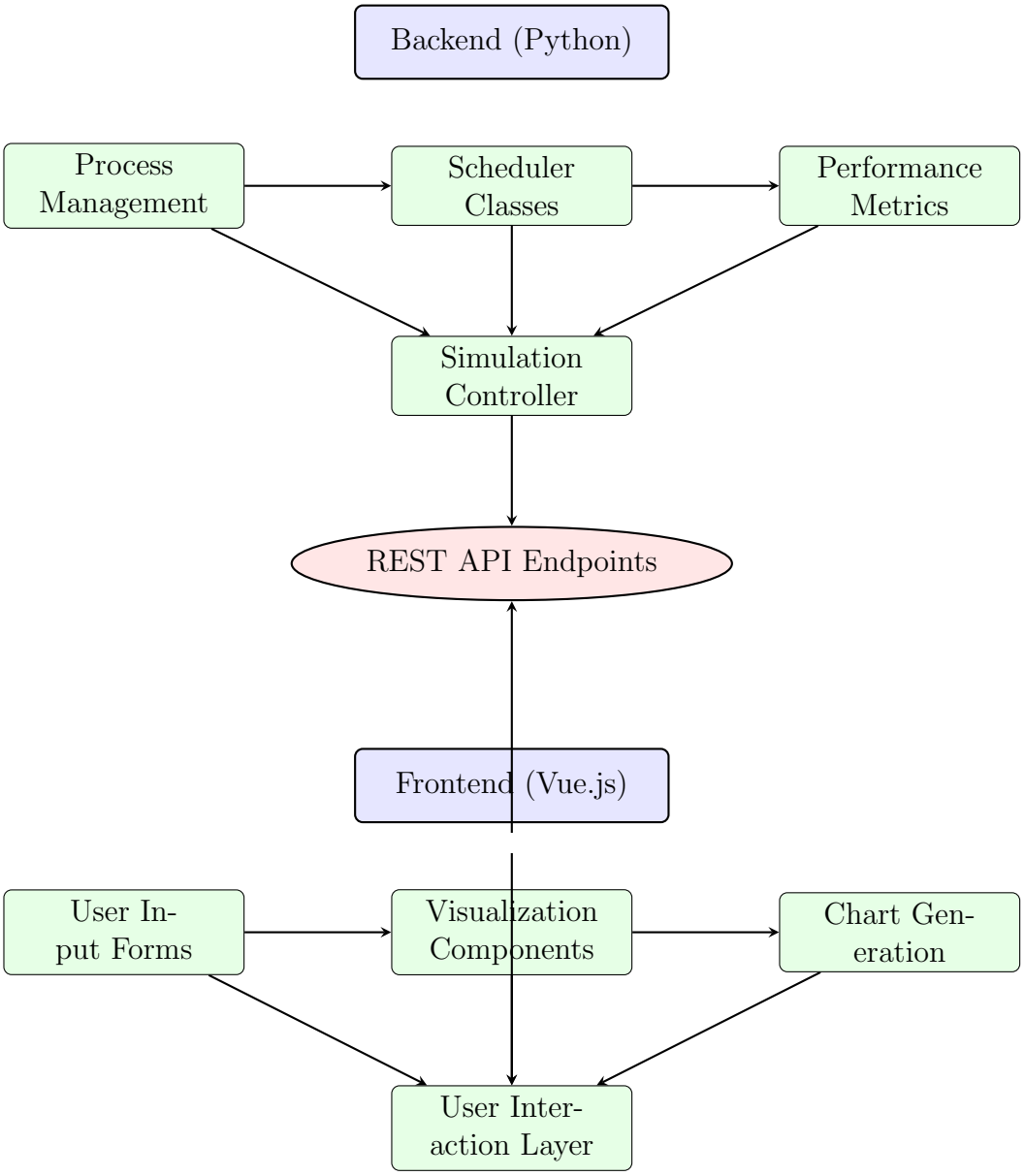

Figure 2.2: Condensed Component Interaction between Frontend and Backend

This diagram illustrates the interaction between the system components. The backend Python modules handle the computation and simulation logic, while the frontend Vue.js components manage user interaction and visualization. Communication between the two layers occurs through REST API endpoints using HTTP requests with JSON payloads.

2.2 Backend Architecture

The backend is implemented in Python and follows object-oriented design principles. It consists of:

- Core classes for process representation and management
- Abstract scheduler class defining common functionality
- Concrete scheduler implementations for each algorithm
- Controller class managing the simulation flow
- API endpoints for frontend communication

2.3 Frontend Architecture

The frontend is built using Vue.js and provides:

- Responsive user interface for simulation interaction
- Process creation and management components
- Algorithm selection and parameter configuration
- Visualization components (Gantt charts, timelines)
- Performance metrics displays and comparison tools

Chapter 3

Design Choices

3.1 Object-Oriented Design

We adopted an object-oriented approach for the scheduling algorithms, using the Strategy pattern through an abstract Scheduler base class with concrete implementations for each algorithm. This design:

- Provides a uniform interface for all scheduling algorithms
- Facilitates adding new algorithms without modifying existing code
- Enables reuse of common functionality across algorithms
- Ensures consistent handling of processes and metrics

3.2 Separation of Concerns

The system separates process management, scheduling logic, and visualization:

- Process class manages individual process attributes and state
- Scheduler classes focus exclusively on algorithm implementation
- Controller class coordinates execution and result collection
- Frontend components handle user interaction and visualization

3.3 State Tracking and Simulation History

To enable step-by-step visualization, we implemented a comprehensive state tracking mechanism:

- Each algorithm records its state at significant execution steps
- States include ready queue contents, running process, and time information
- History can be replayed for educational purposes
- Interactive visualization uses this data for detailed exploration

Chapter 4

Implementation Details

4.1 Process Representation

Each process is represented by the `Process` class with the following attributes:

- Process ID: Unique identifier
- Arrival Time: When the process enters the system
- Burst Time: CPU time required for execution
- Priority: Importance value (lower value means higher priority)
- State variables: Remaining time, start time, completion time, etc.

Mathematically, a process P_i can be represented as:

$$P_i = (id_i, arrival_i, burst_i, priority_i, remaining_i, start_i, completion_i) \quad (4.1)$$

where:

$$waiting_i = completion_i - arrival_i - burst_i \quad (4.2)$$
$$turnaround_i = completion_i - arrival_i \quad (4.3)$$
$$response_i = start_i - arrival_i \quad (4.4)$$

4.2 Scheduler Base Class

The abstract `Scheduler` class provides common functionality:

- Process list management

- Time tracking
- Gantt chart creation
- Performance metric calculations
- Simulation state recording

4.3 Algorithm Implementations

4.3.1 First-Come First-Served (FCFS)

FCFS executes processes in the order they arrive:

- Non-preemptive scheduling
- Sorts processes by arrival time
- Simple implementation with a FIFO queue
- Records state changes when processes arrive and complete

Formally, FCFS scheduling can be defined as:

$$\text{Given processes } P = \{P_1, P_2, \ldots, P_n\} \tag{4.5}$$

$$\text{Order by } arrival_i \text{ such that} \tag{4.6}$$

$$\forall i, j \in \{1, 2, \ldots, n\}, i < j \implies arrival_i \leq arrival_j \tag{4.7}$$

Algorithm 1 FCFS Algorithm

1: Sort processes by arrival time process in sorted order
2: Execute process until completion
3:

4.3.2 Shortest Job First (SJF)

SJF executes the process with the shortest burst time first:

- Non-preemptive implementation
- Maintains a priority queue sorted by burst time
- Records state changes at process selection and completion

Let $R(t)$ be the set of ready processes at time t. SJF selects process P_i such that:

$$P_i \in R(t) \wedge \forall P_j \in R(t) : burst_i \leq burst_j \tag{4.8}$$

Algorithm 2 SJF Algorithm

```
1: while not all processes finished do
2:     Select process with smallest burst time in R(t)
3:     Execute until completion
4: end while
```

4.3.3 Priority Scheduling

Priority scheduling executes processes based on priority value:

- Non-preemptive implementation
- Maintains a priority queue sorted by priority value
- Lower numerical value indicates higher priority
- Records state changes at priority-based selection points

Let $R(t)$ be the set of ready processes at time t. Priority Scheduling selects process P_i such that:

$$P_i \in R(t) \wedge \forall P_j \in R(t) : priority_i \leq priority_j \tag{4.9}$$

Algorithm 3 Priority Scheduling Algorithm

```
1: while not all processes finished do
2:     Select process with highest priority in R(t)
3:     Execute until completion
4: end while
```

4.3.4 Round Robin (RR)

Round Robin allocates a fixed time quantum q to each process:

- Preemptive scheduling with configurable time quantum
- Circular queue implementation

- Records state changes at context switches and quantum expirations
- Tracks remaining burst time for each process

For each time slice of length q:

$$execution\_time_i = \min(remaining_i, q) \tag{4.10}$$

$$remaining_i = remaining_i - execution\_time_i \tag{4.11}$$

If $remaining_i > 0$, the process is placed back in the ready queue.

Algorithm 4 Round Robin Algorithm

```
1: while not all processes finished do process in ready queue
2:     Execute for up to q time units
3:     if burst not finished then
4:         Put process back in queue with updated remaining time
5:     end if
6:
7: end while
```

4.3.5 Priority Round Robin (PRR)

Priority Round Robin combines priority scheduling with round robin:

- Processes are grouped by priority level
- Round Robin is applied within each priority group
- Multiple queue implementation with highest priority served first
- Records detailed state of all priority queues

PRR can be formally described as:

$$\text{Let } Q_p = \{P_i \in R(t) \mid priority_i = p\} \tag{4.12}$$

$$\text{Execute processes in } Q_{p_{min}} \text{ using Round Robin with quantum } q \tag{4.13}$$

$$\text{If } Q_{p_{min}} = \emptyset, \text{ move to } Q_{p_{min}+1} \tag{4.14}$$

Algorithm 5 Priority Round Robin Algorithm

```
while not all processes finished do
    for priority level from highest to lowest do
        while there are processes in Q_p do
            Execute each process using RR with quantum q
        end while
    end for
end while
```

4.4 Performance Metrics Calculation

The simulation calculates standard CPU scheduling performance metrics:

$$\text{Average Waiting Time} = \frac{1}{n}\sum_{i=1}^{n} waiting_i \tag{4.15}$$

$$\text{Average Turnaround Time} = \frac{1}{n}\sum_{i=1}^{n} turnaround_i \tag{4.16}$$

$$\text{Average Response Time} = \frac{1}{n}\sum_{i=1}^{n} response_i \tag{4.17}$$

$$\text{CPU Utilization} = \frac{\sum_{i=1}^{n} burst_i}{\max_i(completion_i) - \min_i(arrival_i)} \times 100\% \tag{4.18}$$

$$\text{Throughput} = \frac{n}{\max_i(completion_i) - \min_i(arrival_i)} \tag{4.19}$$

Where n is the total number of processes.

4.5 Visualization Components

The frontend includes specialized visualization components:

- Gantt Chart: Shows process execution timeline
- Process Table: Displays all process attributes and calculated metrics
- Interactive Visualizer: Step-by-step algorithm execution
- Performance Metrics Display: Shows calculated metrics in real-time

Chapter 5

Usage Scenarios

5.1 Educational Use Case

The primary use case is educational, allowing students to:

- Explore scheduling algorithm behavior with different process sets
- Observe how algorithm selection affects performance metrics
- Understand trade-offs between different scheduling approaches
- Learn through interactive visualization and step-by-step execution

5.2 Process Creation

Users can create processes through:

- Manual process specification (ID, arrival time, burst time, priority)
- Random process generation with configurable parameters
- File upload (CSV or JSON format)

5.3 Algorithm Visualization

The system supports two visualization modes:

- Basic mode: Shows complete execution results and metrics
- Interactive mode: Allows stepping through the algorithm execution

5.4 Performance Comparison

Users can compare algorithms by:

- Running multiple algorithms on the same process set
- Viewing side-by-side metric comparisons
- Analyzing Gantt charts to understand execution differences
- Exploring how time quantum affects Round Robin performance

Chapter 6

Testing Methodology

6.1 Unit Testing

Each component was tested individually:

- Process class methods for correct timing calculations
- Individual scheduler algorithm implementations
- Performance metric calculation accuracy

6.2 Integration Testing

Integration tests verified the interaction between components:

- Controller integration with scheduler implementations
- API endpoint functionality with frontend requests
- End-to-end simulation flow

6.3 Test Cases

Standard test cases included:

6.3.1 Test Case 1: Basic Process Set

Process ID	Arrival Time	Burst Time	Priority
P1	0	5	3
P2	1	3	1
P3	2	8	2
P4	3	2	4

6.3.2 Test Case 2: Simultaneous Arrival

Process ID	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	0	4	1
P3	0	3	2
P4	0	5	4

6.3.3 Test Case 3: Priority Evaluation

Process ID	Arrival Time	Burst Time	Priority
P1	0	5	4
P2	1	7	2
P3	2	3	1
P4	3	4	3

6.3.4 Test Case 4: Round Robin Evaluation

Process ID	Arrival Time	Burst Time
P1	0	10
P2	0	5
P3	0	8

Tested with time quantum values: 1, 2, 4, and 8

6.4 Edge Cases

We tested the following edge cases:

- Empty process list
- Single process execution
- Processes with zero burst time
- Late-arriving high-priority processes

- Very large process sets (¿100 processes)
- Extremely small and large time quantum values

Chapter 7

Performance Analysis

7.1 Comparison Methodology

To compare algorithm performance, we:

- Used identical process sets across all algorithms
- Varied process characteristics (arrival patterns, burst times, priorities)
- Measured standard performance metrics
- Evaluated impact of time quantum on Round Robin variants

7.2 Results

7.2.1 Average Waiting Time

SJF generally provided the lowest average waiting time, as expected from theoretical analysis. FCFS performed worst with varying burst times, while Round Robin's performance depended heavily on time quantum selection.

For a set of n processes, the theoretical lower bound on average waiting time is achieved by SJF and can be approximated as:

$$\text{AWT}_{SJF} \approx \frac{\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \min(burst_i, burst_j)}{n} \tag{7.1}$$

7.2.2 Average Turnaround Time

Similar to waiting time, SJF typically showed the best turnaround time. Priority scheduling performed well when high-priority processes had short burst times.

7.2.3 CPU Utilization

All non-preemptive algorithms (FCFS, SJF, Priority) showed similar CPU utilization. Round Robin variants had slightly lower utilization due to context switching overhead, which can be modeled as:

$$\text{Utilization}_{RR} \approx \frac{\sum_{i=1}^{n} burst_i}{\sum_{i=1}^{n} burst_i + k \times \text{context\_switch\_time}} \tag{7.2}$$

where k is the number of context switches.

7.2.4 Time Quantum Effects

For Round Robin and Priority Round Robin:

The average waiting time in Round Robin can be approximately modeled as a function of time quantum q:

$$\text{AWT}_{RR}(q) \approx \text{AWT}_{FCFS} - \frac{q}{2} \times \frac{n-1}{n} \times (1 - \frac{1}{\bar{b}/q}) \tag{7.3}$$

where $\bar{b}$ is the average burst time.

- Small time quantum ($q \ll \bar{b}$): Increased responsiveness but reduced efficiency due to context switching
- Large time quantum ($q \gg \bar{b}$): Improved efficiency but reduced fairness, approaching FCFS behavior
- Optimal time quantum: Empirically found to be around $q \approx \bar{b}/2$

7.3 Algorithm Strengths and Weaknesses

7.3.1 FCFS

- Strengths: Simplicity, fairness in arrival order, no starvation
- Weaknesses: Convoy effect, poor average waiting time with varying burst times

7.3.2 SJF

- Strengths: Optimal average waiting time, good for batch systems
- Weaknesses: Potential starvation of longer jobs, requires known burst times

7.3.3 Priority Scheduling

- Strengths: Importance-based execution, good for systems with critical tasks
- Weaknesses: Starvation of low-priority processes, priority inversion problems

7.3.4 Round Robin

- Strengths: Fair CPU distribution, good response time, no starvation
- Weaknesses: Higher context switching overhead, performance sensitive to time quantum

7.3.5 Priority Round Robin

- Strengths: Combines benefits of Priority and Round Robin, flexible
- Weaknesses: Complex implementation, requires careful parameter tuning

Chapter 8

Conclusion

8.1 Summary of Findings

Our simulation confirmed theoretical principles of CPU scheduling:

- SJF provides optimal average waiting time but is impractical in real systems
- FCFS is simple but can lead to poor performance with mixed process types
- Priority scheduling handles different process importance but risks starvation
- Round Robin provides fairness and responsiveness at the cost of efficiency
- Priority Round Robin offers a good compromise for many scenarios

The mathematical analysis revealed that:

$$\text{AWT}_{SJF} \leq \text{AWT}_{Priority} \leq \text{AWT}_{FCFS} \tag{8.1}$$

And for Round Robin, the relation between time quantum q and performance follows:

$$\lim_{q \to \infty} \text{AWT}_{RR}(q) = \text{AWT}_{FCFS} \tag{8.2}$$

$$\lim_{q \to 0} \text{AWT}_{RR}(q) = \infty \text{ (due to context switching overhead)} \tag{8.3}$$

8.2 Future Improvements

Potential enhancements for the simulation include:

- Implementation of additional algorithms (Multilevel Feedback Queue with decay function $\alpha(t) = \frac{1}{1+t}$)
- Stochastic process models with burst time distribution $B \sim N(\mu, \sigma^2)$
- Real-time visualization with dynamic parameter adjustment
- Machine learning models for predicting optimal scheduling parameters

8.3 Final Thoughts

This simulation demonstrates the fundamental trade-offs in CPU scheduling algorithms, confirming the theoretical time complexity and performance characteristics. No algorithm provides an optimal solution across all scenarios, making scheduling policy selection context-dependent.